In-Class Exercise/Demo

Exercise Demo

- In today's demo, we will practice CRUD functions using Restful API.
- First, download the **m5-w3-d2-demo .zip** file from GAP Week 3 Day 2.
- Unzip and drop this folder **m5-w3-d2-demo** inside Westcliff folder.
- Launch VS Code and scaffold a react app in here:

  npx create-react-app crud-json-server

- After this, move the **App.js** (to replace the default App.js) and **db.json** files into the **src** and **root** directory respectively of crud-json-server folder.
- CD to crud-json-server.
- Open this db.json file to see what's in here. It's a list of array object data but in json format. Remember: properties in json needs to be quoted unlike js format.

Exercise Demo

- In this demo, we will use a local server to store data – **json server** to perform our Restful API service. Not to be used in a deployment setting but useful for quick development purposes.
- Json server is a ready made Database and a HTTP Server. This package uses a single file db.json (or any other JSON file), treats the file as a JSON database and also exposes routes on which you can GET, POST, PUT, DELETE for that db.json file. To start Json server, you normally have to run json-server --watch db.json --port #### on a terminal.
- Instead of running json server (json-server --watch db.json) and the app (npm start) on separate terminals, we will use **concurrently package** to help us run both concurrently:

  npm i -D json-server concurrently

- After finished installing, we'll head to the package.json file. Notice the new devDependencies:

```
"devDependencies": {
  "concurrently": "^6.1.0",
  "json-server": "^0.16.3"
}
```

Exercise Demo

- In the scripts section of our package.json file, insert the following:

"json-server": "json-server --watch db.json --port 5000",
"dev": "concurrently \"npm start\" \"npm run json-server\""

```
"scripts": {
  "start": "react-scripts start",
  "json-server": "json-server --watch db.json --port 5000",
  "dev": "concurrently \"npm start\" \"npm run json-server\"",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "eject": "react-scripts eject"
},
```

Exercise Demo

- As you can see, npm start and the json-server command was combined into “dev”.
- Now that the json server is installed, we will proceed to create our app.
- Open App.js, notice the App class component already have some local states defined in the constructor.
- In the return(), we will create a button to output the list from the json database.

Exercise Demo

- Create this button within the container, after the title bar:

```
render() {
    return (
        <div className="container">
            <span className="title-bar">
                <button
                    type="button"
                    className="btn btn-primary"
                >
                    Get Lists
                </button>
            </span>
        </div>
    )
}
```

Exercise Demo

- Next, create a new file **Lists.js** in the src folder.
- On this file, add the React import and the Lists functional component:

```
import React from 'react'

function Lists(props) {
    let listrows = [];
    props.alldata.forEach(element => {

    })
    return ()
}
```

- Within this component, we want to display all the json data in a table format. We will use the array prototype method forEach() to loop through each json object and store in the new array listrows[ ].

Exercise Demo

- Next, we'll output each data properties that's looped through and push to the array:

```
props.alldata.forEach(element => {
    listrows.push(
        <tr key={element.id}>
            <td>{element.id}</td>
            <td>{element.title}</td>
            <td>{element.author}</td>
        </tr>
    )
})
```

- In the return() below, we'll create the jsx table:

```
        </tr>
    )
})
return (
    <table className="table table-striped">
        <thead>
            <tr>
                <th>#</th>
                <th>Title</th>
                <th>Author</th>
                <th>Update</th>
                <th>Delete</th>
            </tr>
        </thead>
        <tbody>{listrows}</tbody>
    </table>
)
```

Exercise Demo

- And finally on this file, export the component:

```
        </table>
    )
}

export default Lists;
```

- And install Bootstrap:

  npm install bootstrap

- Then import into Lists.js:

```
import React from 'react'
import "bootstrap/dist/css/bootstrap.min.css";
```

- Next, let's return to App.js and add the onClick event to the button in the render:

```
<button
    type="button"
    className="btn btn-primary"
    onClick={this.getLists}
>
    Get Lists
</button>
```

Exercise Demo

- We'll need to create the handler that's called from the onClick event. We will add this below the constructor:

```
getLists = () => {

}

render() {
    return (
```

- Within this handler, we'll use the Fetch API to fetch the json data object through the port as defined in the package.json file:

```
getLists = () => {
    fetch("http://localhost:5000/posts")
}
```

Exercise Demo

- Next, add the following below fetch():

```
getLists = () => {
    fetch("http://localhost:5000/posts")
    .then(res => res.json())
    .then(result =>
      this.setState({
        loading: false,
        alldata: result
      })
    )
    .catch(console.log);
}
```

Exercise Demo

To explain the previous step:

- The Fetch API comes with return promises containing the response (a Response object). This is just an HTTP response, not the actual JSON. To extract the JSON body content from the response, we use the json() method. (https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch).
- We also then check if the Json content is in the response – if it is, then turn the loading off (false) and set the Json content to our state array property alldata. And finally the catch is to catch any errors.
- So far, we have a Lists component, a button with an onClick event to call the handler and the handler itself all completed and ready to go. However, Lists component is yet to be called. As you recalled, child components are typically called in the parent component – in this case App.js.

Exercise Demo

- We can simply just call the component but in our case, we want the loading component to show while waiting for the JSON data to load. We will create a condition statement (ternary operator) to test which is the case: data loading or loaded:

```
render() {
    const listTable = this.state.loading ? () : ();
    return (
```

- If data loading…

```
const listTable = this.state.loading ? (
    <span>Loading Data......Please be patience.</span>
) : ();
```

Exercise Demo

- If data loaded…we'll call the Lists component and pass the state data containing JSON data to it:

```
const listTable = this.state.loading ? (
    <span>Loading Data......Please be patience.</span>
) : (
    <Lists alldata={this.state.alldata} />
);
```

- With that done, we now need to render/call this loading variable component. Render this within the return():

```
            </button>
        </span>
        {listTable}
    </div>
```

Exercise Demo

- You probably notice an error message in the terminal that the Lists component is not defined. We need to import Lists into App.js:

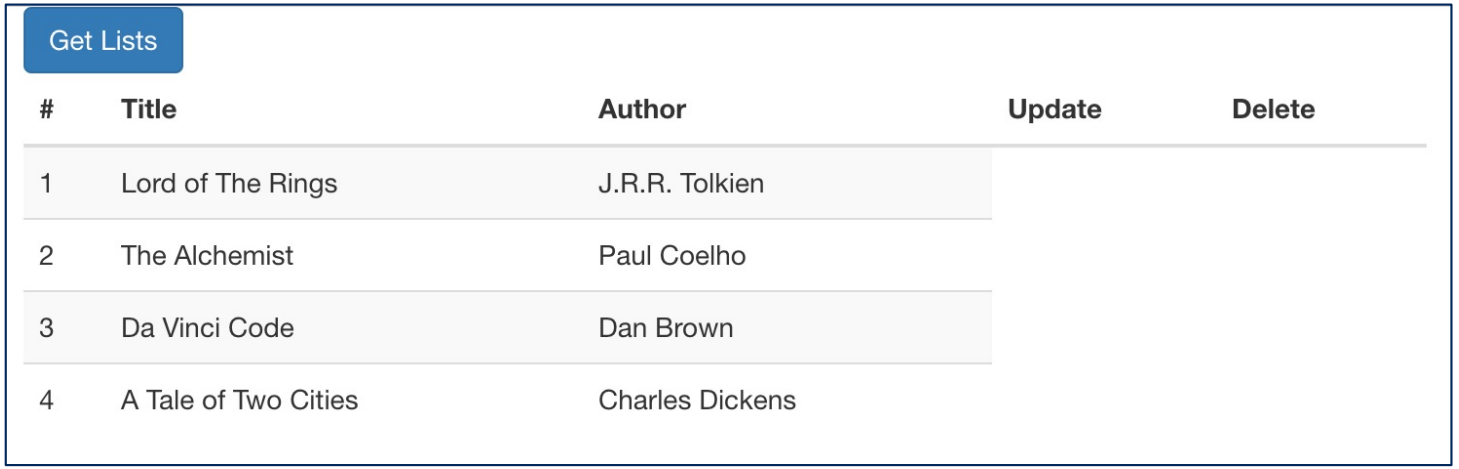
```
import React from "react";
import Lists from "./Lists";
```

- The initial setup is complete. It's time to give it a test drive. On a bash terminal, enter this run both JSON server and the app concurrently:

npm run dev

Exercise Demo

- Click the Get Lists button:



- This app to be continued in your next homework.

Exercise Demo

- **DUE:**

  Today 10.30PM PT.

- **Submission:**

  Post GitHub URL on GAP Week 3 Day 2 Exercise